UADE

UADE

Programación III – Diseño y Análisis de Algoritmos

Temas a desarrollar

1. Repaso Clase Anterior
2. Resolución Ejercicios
3. Estrategia primer parcial
4. Subsecuencia común más larga

Repaso Programación Dinámica

Programación Dinámica

- La técnica de **programación dinámica** se basa en resolver problemas en función de subproblemas más pequeños del mismo tipo.

- El **principio de optimalidad** es el pilar de la técnica y dice que una solución puede ser óptima solo si está basada en soluciones parciales también óptimas.

- La resolución de plantea en forma *bottom-up* a partir de problemas triviales óptimos.

- La técnica se utiliza principalmente para problema de **optimización**.

Programación dinámica

En general se puede decir que los pasos a seguir para plantear una solución por Programación Dinámica son:

Plantear los problemas de tamaño inicial y soluciones directas.

Plantear la solución como una decisión entre soluciones menores.



Síntesis Fibonacci

- La resolución de Fibonacci se basa en ir calculando el valor de Fibonacci ***i*** con base en los valores ya calculados previamente para ***i-1*** e ***i-2***, los cuales fueron guardados y se usan en ese instante.

- La **complejidad temporal** de la solución por programación dinámica es de *O(n)*, la cual es mucho menor comparada con la complejidad temporal $O(2^n)$ que se obtiene si se resuelve por la estrategia de divide y conquista.

Síntesis problema del cambio

- Se tiene un conjunto de ***n*** denominaciones de monedas, de las cuales hay cantidad infinita por cada una de esas denominaciones. Dado un valor ***v*** se debe devolver la menor cantidad de monedad que cubran ese valor.

- El presente enunciado es el mismo que para el problema del cambio visto en *Greedy*, con la condición de que el conjunto de denominaciones debe ser cualquiera y no uno determinado.

- La estrategia por programación dinámica se basa en la siguiente función:

$$C[i,j] \in \begin{cases} 0 & \text{si } j = 0 \\ +\infty & \text{si } i = 1 \text{ y } d_i > j \\ 1 + C[i, j - d_i] & \text{si } i = 1 \text{ y} d_i \leq j \\ c[i-1, j] & \text{si } i > 1 \text{ y } d_i > j \\ \text{mín}(1 + C[i, j - d_i], c[i-1, j]) & \text{sino} \end{cases}$$

- La complejidad temporal es *O(n*v)*, siendo *n* la cantidad de denominaciones y *v* el valor a devolver, versus la solución por Greedy de *O(v)*. Con la diferencia de que por programación dinámica no hay condicionantes en las denominaciones de las monedas, que sí los había en la solución utilizando Greedy.

Síntesis problema de la mochila

- El problema consiste completar una mochila de tamaño P, con objetos que tienen un peso y un valor, maximizando el valor que aportan los objetos ingresados en la mochila. Los objetos no se pueden fraccionar.

- La estrategia de formar la solución a través de solución de problemas menores, basados en el principio de optimalidad se representa como:

$$V[o,p] \in \begin{cases} 0 & \text{si } o = 1 \text{ y } peso[o] > p \\ valor[o] & \text{si } o = 1 \text{ y } peso[o] \leq p \\ V[o-1,p] & \text{si } o > 1 \text{ y } peso[o] > p \\ \text{máx}(V[o-1,p], V[o-1,p-peso[o]] + valor[o]) & \text{sino} \end{cases}$$

- La complejidad temporal es de *O(o * p)*, donde o representa a la cantidad de objetos y *p* a la capacidad de la mochila.

- Si comparamos la solución por Greedy de *O(n log n)*, donde *n* representa la cantidad de objetos, puede llegar a ser más eficiente, pero tiene la limitante que los objetos se deben fraccionar, y en ciertas situaciones eso no va a ser válido.

Revisión Ejercicios Práctica

UADE

Problemas a resolver

2. Howard es un fanático de los comics. En la tienda de comics tienen un único ejemplar de cada revista. Cada revista tiene en la tapa el precio y la cantidad de páginas que contiene. Un día particular Howard decide comprar todas las revistas posibles de modo de maximizar la cantidad total de páginas sin superar su presupuesto disponible de $P. La tienda tiene listados con la información de precio y cantidad de páginas de cada revista ordenados por todos los criterios que sean necesarios. ¿Cuántas páginas en total tendrá si compra la mejor combinación posible de revistas?

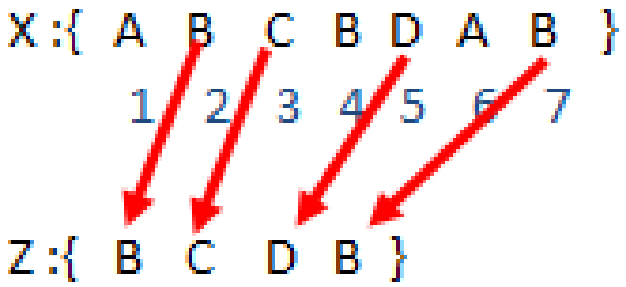
3. Nos vamos de compras un shopping. Tenemos **K** pesos en la billetera para gastar y una lista de **m** productos que podemos comprar. Cada producto tiene un precio, **pi** (que será siempre un número entero), y una utilidad, **ui**. De cada producto podemos comprar como máximo 3 unidades. Además, tenemos una oferta según la cual la segunda unidad de cada producto nos cuesta 1 peso menos, y la tercera 2 pesos menos del valor original de ese producto. Queremos elegir los productos a comprar, maximizando la utilidad de los productos comprados.

Subsecuencia común más larga

Subsecuencia común más larga

Qué es una subsecuencia: “Dada una secuencia *X ={ x1; x2; …; xm}*, decimos que *Z = {z1; z2; ; zk}* es una subsecuencia de *X (siendo <= m)* si existe una secuencia creciente *{i1; i2; ; ik}* de índices de *X* tales que para todo *j = 1; 2; …; k tenemos xij = zj* .”

Por ejemplo, *Z = {BCDB}* es una subsecuencia de *X = {ABCBDAB}* con la correspondiente secuencia de índices *{2; 3; 5; 7}*, dada por:

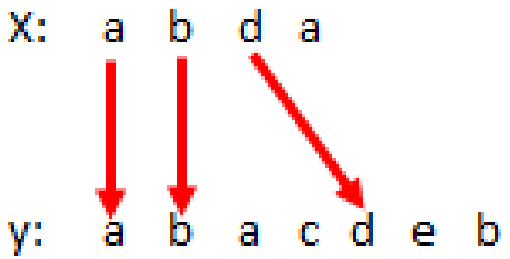


UADE

Subsecuencia común más larga

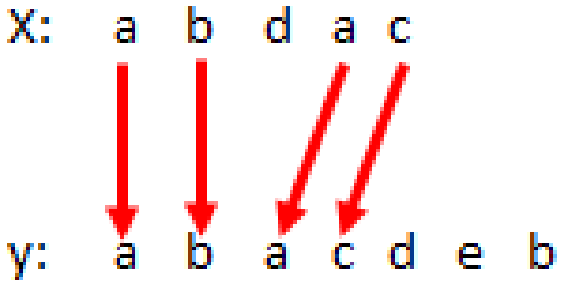
Problema de la **Subsecuencia común más larga**: “Dadas dos secuencias *X* e *Y*, decimos que *Z* es una subsecuencia común de *X* e *Y* si es subsecuencia de *X* y subsecuencia de *Y* . Se desea encontrar la longitud de la subsecuencia de longitud máxima común a dos secuencias dadas”.

Por ejemplo 1: si tenemos *X* = ***abda*** e *Y* = ***abacdeb*** la subsecuencia común más larga es de longitud 3.



Subsecuencia común más larga

Ejemplo 2: si tenemos *X* = ***abdac*** e *Y* = ***abacdeb*** la subsecuencia común más larga es de longitud 4.



Subsecuencia común más larga - Estrategia

La solución a este problema estará dada por considerar soluciones parciales en las que se consideran los primeros *i* caracteres de la secuencia *X* y los primeros *j* caracteres de la secuencia *Y*, generando la siguiente función:

$$L[i,j] \in \begin{cases} 0 & \text{si } i = 0 \text{ o } j = 0 \\ L[i-1, j-1] + 1 & \text{si } i > 0 \text{ y } j > 0 \text{ y } X_i = Y_j \\ \max(L[i-1,j], L[i,j-1]) & \text{sino} \end{cases}$$

		a	b	a	c	d	e	b
	0	0	0	0	0	0	0	0
a	0	1	1	1	1	1	1	1
b	0	1	2	2	2	2	2	2
d	0	1	2	2	2	3	3	3
a	0	1	2	3	3	3	3	3
c	0	1	2	3	4	4	4	4

Subsecuencia común más larga - Algoritmo

```
ALGORITMO SUBSECUENCIAMAXIMA
Entrada: X, Y: Vector<Caracter>
Salida: v: entero
  M ← inicializarMatrix(longitud(X) + 1, longitud(Y) + 1)
  para i = 0 hasta longitud(X)
    para j = 0 hasta longitud(Y)
      si i = 0 O j = 0
        M[i, j] ← 0
      sino
        si X[i] = Y[j]
          M[i, j] ← M[i − 1, j − 1] + 1
        sino
          M[i, j] ← máx(M[i − 1, j], M[i, j − 1])
        fin si
      fin si
    fin para
  fin para
  devolver M[longitud(X), longitud(Y)]
```

$$L[i,j] \in \begin{cases} 0 & \text{si } i = 0 \text{ o } j = 0 \\ L[i-1, j-1] + 1 & \text{si } i > 0 \text{ y } j > 0 \text{ y } X_i = Y_j \\ \text{máx}(L[i-1, j], L[i, j-1]) & \text{sino} \end{cases}$$

Subs. común más larga – Complejidad temporal

```
ALGORITMO SUBSECUENCIAMAXIMA
Entrada: X, Y: Vector<Caracter>
Salida: v: entero
  M ← inicializarMatrix(longitud(X) + 1, longitud(Y) + 1)
  para i = 0 hasta longitud(X)
    para j = 0 hasta longitud(Y)
      si i = 0 O j = 0
        M[i, j] ← 0
      sino
        si X[i] = Y[j]
          M[i, j] ← M[i − 1, j − 1] + 1
        sino
          M[i, j] ← máx(M[i − 1, j], M[i, j − 1])
        fin si
      fin si
    fin para
  fin para
  devolver M[longitud(X), longitud(Y)]
```

Θ (X *Y)

¡Muchas gracias!

UADE